

SShion0721/TD_Muzero 深度研究与 SOTA 改造分析报告

执行摘要

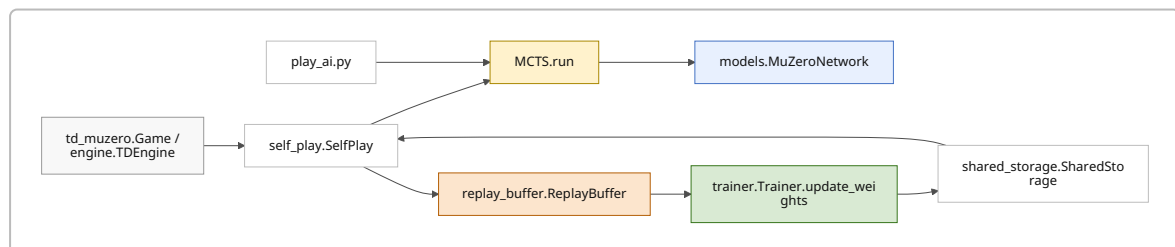
这份仓库本质上是把 `muzero-general` 的教学型实现，接到一个自定义的 11×11 塔防环境上；它已经能“跑起来”，但距离“在实际任务上达到或接近 SOTA、并显著提升训练与推理速度”还有很大工程和算法鸿沟。仓库当前最关键的问题，并不是“网络太小”这么简单，而是**环境建模、动作表示、合法动作处理、训练正确性、搜索实现与系统工程**同时存在短板：环境带有随机性却没有正确播种；MCTS 根节点只用真实合法动作，非根节点却直接展开整个 727 维动作空间；动作在 dynamics 中被编码成“归一化动作编号平面”，几乎没有利用塔防动作的结构；训练代码里还有一个高风险的 Python 闭包式梯度缩放 bug；而配置把 `checkpoint_interval` 设成 5，等于在训练过程中高频复制权重与保存检查点，极可能造成明显的 CPU/IO 开销。 1 2 3 4 5

就“该怎么把它改到 SOTA”而言，我的结论是：**不要直接在现有代码上先堆大模型**。更合理的路线是分两阶段推进。第一阶段，先把这个仓库改成一个**正确、可复现、可高速迭代的强基线**：修正种子与训练 bug、重构环境和合法动作生成、把 flat 727 动作改成结构化动作表示、加 AMP/ `torch.compile` / `inference_mode` / profiler、降低 Ray 和 checkpoint 的系统开销。第二阶段，再在这个强基线上引入更接近 SOTA 的 MuZero 家族改进：**EfficientZero/EfficientZero V2 式样本效率改进、Gumbel MuZero 的低模拟数搜索改进、ReZero 的 reanalyze 加速、以及更长期的 UniZero 式 transformer latent world model**。这些方向分别代表了 MuZero 系列在样本效率、搜索质量、训练速度与长时依赖建模上的主流前沿。 6

需要特别说明的是：这个塔防环境是仓库私有定制任务，公开资料里并没有它的标准 leaderboard。因此，这里所谓“SOTA”更适合拆成两层目标：一层是在**标准 MuZero benchmark** 上接近或复现实证最强开源结果；另一层是在**自定义 Tower Defense 任务**上显著超过当前仓库基线，并且同时超过 PPO / DreamerV3 / LightZero-MuZero / EfficientZero 等强基线。对这个仓库而言，最现实的高收益组合是：**环境与动作重构 + 工程提速 + Gumbel/ReZero + EfficientZero-lite**；若追求更高上限，再进入 **UniZero/Transformer** 路径。 7

仓库现状与关键问题审查

从结构上看，这个仓库包含两层代码：一层是上游 `muzero-general` 风格的通用 MuZero 框架，核心文件包括 `muzero-general/muzero.py`、`trainer.py`、`self_play.py`、`replay_buffer.py`、`shared_storage.py`、`models.py`；另一层是这个项目自己的塔防环境实现与包装，包括 `muzero-general/games/td_muzero.py`、`muzero-general/games/engine.py`，以及一个独立的可视化推理脚本 `play_ai.py`。上游 README 也明确说明该基础实现“primarily for educational purpose”，并且“不系统追求人类水平或最佳超参”，这意味着它天然更适合作为研究起点，而不是直接作为高性能生产线。 8 3 4 9 10 1 2 11 12



当前任务配置在 `muzero-general/games/td_muzero.py` 中写死：观测为 `(5, 11, 11)`，动作空间大小为 `11*11*6+1=727`，单玩家、`max_moves=100`，网络类型为 `modern`，仅 2 个 block、32 个 channels，`num_simulations=30`，`num_unroll_steps=10`，`td_steps=10`，启用了 PER 与 `reanalyse`，优化器为 Adam，学习率 `0.003`，并把 `checkpoint_interval` 设成了 `5`。这套配置更像一个“可启动实验”的轻量试配，而不是面向高分数、高吞吐或复杂塔防时序的认真配置。¹

环境侧存在几个会直接影响性能上限与复现性的设计问题。第一，塔防环境其实是**随机环境**：`engine.py` 里的 `_get_wave_enemies()` 会调用 `random.shuffle(enemies_format)` 打乱波次出怪顺序；但 `Game(seed)` 并没有把种子传给环境，`TDEngine` 也没有保存或使用外部种子。这意味着配置中的 `seed` 并没有真正控制环境随机性。第二，`reset()` 里对 `wave` 和 `enemies_to_spawn` 连续执行了两遍初始化，第一遍生成的随机波次会立刻被第二遍覆盖，平白增加一次随机打乱与 CPU 开销。第三，环境 `get_state()` 明明包含 `wave`、`tower type`、`cooldown` 等信息，但 `get_observation()` 最终只使用了 5 个通道：塔占位、出生点/基地、敌人 HP、基地 HP、金钱，完全丢掉了 tower 类型、等级、冷却、当前 wave、spawn timer、敌人速度/类型等关键信息。对一个强依赖“建造顺序、塔种类搭配、敌人波次特征”的塔防任务来说，这相当于故意给模型上了信息瓶颈。^{2 1}

模型侧最大的结构性问题有两个。第一，当前动作空间是扁平的 727 分类头，但塔防动作天然就是结构化的：`(action_type, x, y)` 或 `wait`。仓库没有利用这点。第二，更严重的是，`models.py` 的 `dynamics` 分支里，所谓 `action_one_hot` 实际并不是 one-hot，而是把动作编号除以动作空间大小后，广播成一个常数平面，再与 hidden state 拼接。这种编码方式对“在某个格子建某种塔、升级哪一座塔、卖哪一座塔”几乎没有归纳偏置，既不表达二维位置局部性，也不表达动作类型语义。对于这种强结构化离散动作空间，这是非常不利的设计。与此同时，`td_muzero.py` 里把 active backbone 设为 `modern`，但规模只有 32 channels / 2 blocks，本身也偏小。^{5 1}

搜索侧则存在一个会显著拖累规划质量的核心问题：根节点扩展时，`self_play.py::MCTS.run` 使用了真实环境给出的 `legal_actions`；但一旦进入 latent tree，代码会在 recurrent inference 后直接用 `self.config.action_space` 进行 `node.expand(...)`，也就是**非根节点默认所有 727 个动作都可用**。这在 Atari 一类“所有按钮都一直合法”的环境问题不大，但在塔防这种**状态相关合法动作**极强的环境里，会使搜索在隐空间中大量考虑“根本不合法”的建塔/升级/出售动作。虽然真实环境里非法动作会给 `-5` 惩罚，模型最终可能学会一部分，但规划先验与搜索空间已经被污染了。`play_ai.py` 里复制的最小 MCTS 也有同样问题。^{4 11}

训练正确性方面，还有一个高风险 bug 值得优先修。`trainer.py::update_weights` 在 unroll loss 循环中多次使用 `register_hook(lambda grad: grad / gradient_scale_batch[:, i])`。按 Python 闭包的 late binding 语义，这个 `i` 在 backward 真正执行时，很可能已经变成循环结束后的最终值，也就是所有 step 的 hook 都共享同一个 `i`。更稳妥的写法应该是用默认参数绑定当前索引，例如 `lambda grad, j=i: grad / gradient_scale_batch[:, j]`。如果这一点没有被运行时偶然规避，那么当前实现可能把所有 unroll step 的梯度缩放都用错了。³

训练与评估流程整体上也相当原始。`trainer.py` 每个 batch 都从 Python list / NumPy array 重新构造 tensor；没有 DataLoader、没有 pinned memory、没有 AMP、没有 `torch.compile`、没有 gradient checkpointing。`self_play.py` 和 `play_ai.py` 的搜索完全是 Python 循环；MCTS 没有 batch leaf expansion，也没有 C++/Cython tree。评估指标基本只有 `total_reward`、`mean_value`、`episode_length`，并通过 `TensorBoard` 写入；没有多 seed 置信区间，没有显式 wall-clock 吞吐指标，没有 invalid-action rate、average wave、base HP、MCTS ms/move、GPU utilization 之类真正能指导研发的指标。^{3 4 8}

最后，系统瓶颈从静态分析上看也很明确。`legal_actions()` 会遍历所有格子与塔类型，并频繁调用 `can_place_tower()`；后者每次又要临时修改网格并运行 `A.find_path()` 检查路径是否仍连通。也就是说，每一步决策都在 CPU 上做大量 Python 循环 + 路径搜索*。这类任务在 MuZero 中又会被 MCTS 成倍放大，很可

能成为主瓶颈。再叠加 Ray 轮询式 `ray.get(...)`、`time.sleep(...)`、高频 checkpoint、旧版本 `ray==1.5.2` / `torch==1.10.0` 依赖，仓库当前的速度上限不会高。Ray 官方文档也明确提醒：小任务会因调度开销难以提速，过于频繁的 `ray.get` 会伤害并行性；并建议用 Timeline、Dashboard CPU flame graph、PyTorch Profiler 和 Nsight 做定位。 [1](#) [2](#) [3](#) [4](#) [13](#) [14](#)

当前仓库审查摘要表

维度	当前实现	高风险点	直接影响	关键证据
代码基座	基于 <code>muzero-general</code> 教学实现	不是高性能基座	需要自行补齐 benchmark / profiling / speed path	12
环境随机性	<code>random.shuffle</code> 出怪，但未正确播种	不可复现，且 deterministic MuZero 与任务不匹配	baseline 噪声大，难做显著性比较	2 1
观测设计	只有 5 通道	丢掉 tower type/ level/cooldown/ wave 等信息	学习难、长时依赖弱	1 2
动作表示	727-way flat policy, dynamics 用归一化动作编号平面	缺乏位置与动作类型归纳偏置	样本效率与规划质量下降	5
合法动作处理	根节点用真实 <code>legal_actions</code> ，潜在节点展开全动作空间	隐空间搜索污染严重	MCTS 会搜索大量非法动作	4 11
训练正确性	<code>register_hook(lambda ... i)</code>	存在闭包晚绑定风险	unroll 梯度缩放可能错误	3
系统工程	<code>checkpoint_interval=5</code> ，Ray 轮询，旧版 torch/ray	CPU/IO/Ray 开销大	wall-clock 训练慢	1 3 13
评估	只记 total reward 等少数指标	缺少多 seed、CI、速度指标	无法严谨声称“接近 SOTA”	8

MuZero 与相关 SOTA 方向对比

MuZero 本身是“value-equivalent latent model + MCTS”的里程碑：它在不知道规则的情况下，在 Go、Chess、Shogi 与 57 个 Atari 游戏上达到超人或当时 SOTA 水平。随后几年，MuZero 系列的主要进化路线比较清晰：**复杂动作空间**由 Sampled MuZero 扩展；**低模拟数搜索质量**由 Gumbel MuZero 改进；**随机环境**由 Stochastic MuZero 解决；**样本效率**由 EfficientZero / EfficientZero V2 推高；**系统与算法统一基座**由 LightZero 建立；**长时依赖与可扩展世界模型**则由 UniZero 推进；**reanalyse 墙钟时间**由 ReZero 重点优化。

[15](#)

近期 MuZero 相关论文与实现比较

论文	作者	年份	关键改进点	开源实现	适用场景
Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model	Schrittwieser et al.	2019	原始 MuZero; 隐空间表示/动力学/预测三分支; MCTS + value-equivalent model	仅伪代码官方公开; 常用开源复现为 <code>muzero-general</code> 、MiniZero 等 ¹⁶	离散动作、规划可帮助学习的任务
Learning and Planning in Complex Action Spaces	Hubert et al.	2021	Sampled MuZero; 在复杂/连续动作空间中对动作进行采样搜索	LightZero 已集成 ¹⁷	大离散、连续、组合动作空间
Policy Improvement by Planning with Gumbel	Danihelka et al.	2022	Gumbel MuZero; Gumbel-Top-k + Sequential Halving, 低模拟数下更稳、更强	MiniZero、LightZero 支持 ¹⁸	动作多、模拟预算小、追求更高搜索性价比
Planning in Stochastic Environments with a Learned Model	Antonoglou et al.	2022	Stochastic MuZero; afterstate + stochastic tree search, 处理随机/部分可观测环境	LightZero 支持 ¹⁹	环境转移随机、观测混叠
Mastering Atari Games with Limited Data	Ye et al.	2021	EfficientZero; Atari 100k 以 194.3% mean / 109.0% median human 达到突破性样本效率	官方仓库公开, 且建议用 torch AMP 加速训练 ²⁰	样本效率敏感的视觉 RL
EfficientZero V2	Wang et al.	2024	将 EfficientZero 扩展到离散/连续、视觉/低维控制; 在 66 个任务里 50 个胜过 DreamerV3	论文公开; 可在 LightZero 生态对照实现 ²¹	想做“强通用 sample-efficient baseline”

论文	作者	年份	关键改进点	开源实现	适用场景
LightZero	Niu et al.	2023	统一 MCTS/RL benchmark; 支持 MuZero、Sampled、Stochastic、EfficientZero、Gumbel、UniZero 等; MCTS 提供 Python/C++ 两套实现, 并强调混合异构加速	官方仓库与中文 README 完整公开 ²²	想快速获得强工程基座与可比 benchmark
UniZero	Pu et al.	2024	Transformer latent world model; 解耦 latent state 与 latent history, 强化长时依赖与多任务扩展	代码在 LightZero 中公开; 文中称单帧输入即可匹配或超过多帧 MuZero-style 算法	长时依赖、记忆型任务、多任务扩展 ²³
ReZero: Boosting MCTS-based Algorithms by Just-in-Time and Speedy Reanalyze	Xuan et al.	2024	减少 reanalyze 调用频次、复用轨迹后续信息加速搜索; 显著提升训练速度同时保持样本效率	代码在 LightZero 中公开 ²⁴	reanalyse 成本高、MCTS 墙钟慢
MiniZero	Wu et al.	2023/2024	统一比较 AlphaZero/ MuZero/ Gumbel; 提出 progressive simulation	官方训练框架公开 ²⁵	想严谨比较搜索预算与算法选择
Stabilizing Transformers for Reinforcement Learning	Parisotto et al.	2019	GTrXL; 面向 RL 的稳定 Transformer 设计, 在记忆任务上超过 LSTM	论文公开 ²⁶	需要长时依赖而又担心普通 Transformer 不稳
Equivariant MuZero	Deac et al.	2023	将环境对称性注入 MuZero world model, 提高数据效率与泛化	论文公开 ²⁷	存在平移/旋转对称性的网格任务

论文	作者	年份	关键改进点	开源实现	适用场景
TransZero	Malmsten & Böhmer	2025	用 transformer 并行扩展子树，报告可达 11× 墙钟加速	代码公开，但仍属较新方向 ²⁸	强烈追求并行搜索与实时性，接受更高研究风险

对这个 TD 仓库，最值得借鉴的不是全部，而是四条最相关主线。**第一，Sampled/Gumbel MuZero** 解决的是“动作多、模拟数小”的问题；你的 727 动作平坦头正处在这一痛点。**第二，Stochastic MuZero** 与你的任务天然相关，因为波次出怪顺序存在随机打乱，而当前代码既没精确播种，也没显式 stochastic modeling。**第三，EfficientZero / V2** 是样本效率最强的现成路线，适合先做“强 baseline”。**第四，UniZero / GTrXL** 对塔防这种长周期回报、波次依赖、建塔历史影响深远的任务很有吸引力，但实现难度和调参复杂度也明显更高。

29

优先级改造方案

高优先级路径

我建议先把改造分成“必做修复”和“高收益升级”两类。前者不做，后面的 SOTA 讨论都没有意义；后者做完，哪怕还没上 transformer，你也会得到一个远强于当前仓库的版本。

必做修复与高收益升级总表

优先级	改动	预期收益	实现难度	主要代码位置
最高	修复环境随机种子、 <code>reset()</code> 重置初始化、训练 hook 闭包 bug	复现性恢复；训练目标更可靠	低	<code>games/engine.py</code> 、 <code>games/td_muzero.py</code> 、 <code>trainer.py</code>
最高	重构合法动作生成与观测	self-play 吞吐大幅提升；学习更稳定	中	<code>games/engine.py</code> 、 <code>games/td_muzero.py</code>
最高	在 latent tree 中加入 legality mask，而非展开全动作空间	MCTS 质量显著改善	中高	<code>self_play.py</code> 、 <code>play_ai.py</code> 、 <code>models.py</code>
高	把 flat 727 动作改成结构化动作头 + 学习式动作嵌入	样本效率、策略质量、推理速度同步提升	中高	<code>models.py</code> 、 <code>trainer.py</code> 、 <code>self_play.py</code>
高	加入 AMP / <code>torch.compile</code> / <code>inference_mode</code> / profiler / checkpoint 策略重构	训练与推理墙钟明显改善	低中	<code>trainer.py</code> 、 <code>self_play.py</code> 、 <code>play_ai.py</code>
高	引入 Gumbel MuZero + ReZero 思想	低模拟数下更强，reanalyse 更快	中	<code>self_play.py</code> 、 <code>replay_buffer.py</code>
中高	引入 EfficientZero-lite 训练范式	样本效率接近主流强 baseline	中高	<code>models.py</code> 、 <code>trainer.py</code> 、 <code>replay_buffer.py</code>

优先级	改动	预期收益	实现难度	主要代码位置
中高	引入 UniZero/GTrXL 式 transformer latent history	长时依赖能力上限更高	高	<code>models.py</code> 、训练全流程
可选	迁移到 LightZero 基座	最快获得强 benchmark 与 C++ MCTS	中高	整体迁移，而非局部 patch

先修正确性与复现性

第一步应立即修正三处代码。

其一，给环境加真正的种子管理，并删除 `reset()` 中重复初始化 `wave/enemies_to_spawn` 的两行。其二，`Game(seed)` 要把 `seed` 向下传给 `TDEngine`。其三，修正 `trainer.py` 中 `register_hook` 的 late binding 风险。当前相关逻辑分别位于 `games/engine.py::_get_wave_enemies` / `reset`、`games/td_muzero.py::Game.__init__` 和 `trainer.py::update_weights`。² ¹ ³

建议直接按下面的补丁思路改：

```
# games/engine.py
import random

class TDEngine:
    def __init__(self, width=11, height=11, seed=0):
        self.width = width
        self.height = height
        self.rng = random.Random(seed)
        self.seed = seed
        self.spawn_x, self.spawn_y = 0, height // 2
        self.base_x, self.base_y = width - 1, height // 2
        self.reset()

    def _get_wave_enemies(self):
        enemies_format = ...
        self.rng.shuffle(enemies_format)
        return enemies_format

    def reset(self):
        ...
        self.wave = 1
        self.enemies_to_spawn = self._get_wave_enemies()
        # 删除重复的第二次赋值
```

```
# games/td_muzero.py
class Game:
```

```
def __init__(self, seed=None):
    self.seed = 0 if seed is None else int(seed)
    self.env = TDEngine(width=11, height=11, seed=self.seed)
```

```
# trainer.py
current_value_loss.register_hook(
    lambda grad, j=i: grad / gradient_scale_batch[:, j]
)
current_reward_loss.register_hook(
    lambda grad, j=i: grad / gradient_scale_batch[:, j]
)
current_policy_loss.register_hook(
    lambda grad, j=i: grad / gradient_scale_batch[:, j]
)
```

这部分的预期收益并不只是“更规范”，而是为所有后续实验打地基。否则你很难判断任何收益是来自算法，还是来自环境随机性与训练实现误差。

重构环境、观测与合法动作

我认为这是**这个仓库最值得先做的改造**。当前塔防环境的信息入口和动作出口都不适合 MuZero。最现实的做法不是马上上 UniZero，而是先把环境接口改成“对搜索和学习友好”。

第一，重做 observation。当前 `get_state()` 已经暴露了很多有用信息，所以最省工程的方法是不改环境规则，只改 `get_observation()`。建议把 5 通道至少扩成以下几类：tower occupancy by type、tower level、tower cooldown、enemy density、enemy hp、enemy speed/类型、当前 wave、spawn timer、money、base hp、距离基地的 distance field、当前合法建造 mask。这样做能显著减少“模型必须从稀疏奖励中自己发明状态抽象”的难度。现有观测只保留了塔占位、敌人 HP、基地血量与资金，信息远远不够。 ¹ ²

第二，合法动作生成不要再在每一步里“按塔类型 × 按格子 × 每格都跑一次路径检查”。更高效又容易保证正确性的方案是：先对每个空格只做一次“如果该格被占住，spawn 到 base 是否仍连通”的 BFS/A 检查，得到一个 placeable cell mask；然后再按塔类型和 affordability 展开 build 动作。也就是说，“路径是否断开”是与 tower type 无关的 cell-level 条件，没必要做 4 次重复检查。11×11 的小图即便每个空格做一次 BFS，成本也会远低于现在的重复 A。这一项往往就是 self-play 的最大加速来源。 ¹ ²

第三，把“wait 1 step”扩成**宏动作**，例如 `wait {1,2,4,8}`。当前动作空间只有一个 wait，并且 `Game.step` 每个动作都只推进 `dt=1.0` 秒。塔防大量时间其实是“等经济、等下一波、等敌人进入射程”，在这些阶段让 MuZero 每秒都做一次完整的 MCTS 非常浪费。宏 wait 动作可以在几乎不伤性能的前提下，把搜索深度和环境交互步数砍掉一大截。当前动作映射逻辑就在 `games/td_muzero.py::step` 与 `action_to_string` 里，改动很集中。 ¹

结构化动作表示与 legality-aware MCTS

这是把任务“做成塔防”而不是“把塔防硬塞给 Atari 式 MuZero”的关键。

最建议的方案，是把动作从 flat 727-way 改成**结构化动作头**。可以先用一个工程上比较稳妥的版本：

`action_type` 头预测 `{build_basic, build_sniper, build_aoe, build_slow, upgrade, sell, wait_k...}`，再用 `coord` 头预测 `(x, y)`，最后组合成合法动作；同时用一个 `validity head` 预测 latent

state 下的 legality mask，在 MCTS 非根节点裁剪非法动作。这样既保留 MuZero 的离散动作接口，又把塔防动作的几何结构暴露给网络。当前仓库并没有任何类似机制，非根节点直接展开全动作空间。 4 11 5

一个可实现的中间态 patch 如下：

```
# models.py
class TDPolicyHead(nn.Module):
    def __init__(self, channels, grid_h=11, grid_w=11, n_types=7):
        super().__init__()
        self.type_head = nn.Linear(channels, n_types)
        self.coord_head = nn.Conv2d(channels, 1, kernel_size=1)

    def forward(self, z):
        pooled = z.mean(dim=(2, 3)) # [B, C]
        type_logits = self.type_head(pooled) # [B, T]
        coord_logits = self.coord_head(z).flatten(1) # [B, H*W]
        return type_logits, coord_logits
```

```
# self_play.py
# root: exact legal mask from env
policy_logits = mask_illegal(policy_logits, legal_mask)

# latent node: use predicted legality head or top-k sampled candidate actions
pred_legal = torch.sigmoid(legality_logits) > 0.5
candidate_actions = decode_structured_actions(type_logits, coord_logits, pred_legal)
node.expand(candidate_actions, ...)
```

动作注入 dynamics 时，我不建议继续使用“动作编号常数平面”。更合理的做法是：为动作类型、x 坐标、y 坐标各自学习 embedding，然后通过 concat 或 FiLM 调制 hidden state。例如：

```
a = type_embed[a_type] + x_embed[x] + y_embed[y]
gamma, beta = film(a).chunk(2, dim=-1)
z = dynamics_backbone(hidden * (1 + gamma[..., None, None]) + beta[..., None, None])
```

预计这项改造会同时带来三个收益：策略头更易学、搜索更少访问垃圾动作、dynamics 更容易学会“位置改变带来的局部后果”。对 11×11 塔防来说，这比盲目把 channel 从 32 提到 256 更重要。

训练与推理基础提速

仓库当前锁定 torch==1.10.0、ray==1.5.2，意味着它直接错过了很多现代 PyTorch 的通用提速手段。官方文档显示，AMP 会让线性层和卷积在低精度下显著更快；torch.compile 会对模型与函数进行编译和缓存；inference_mode 相比 no_grad 还会移除 view tracking 与 version counter 的额外开销；activation checkpointing 能用“重算”换显存；PyTorch Profiler 能直接采集训练/推理中的 CPU/CUDA 时间、内存、shape、stack trace，并导出 TensorBoard trace。 30

因此，这一部分我建议一次性做完：

```
# trainer.py
self.model = models.MuZeroNetwork(self.config).to(device)
self.model = torch.compile(self.model) # 需要升级到 PyTorch 2.x

scaler = torch.amp.GradScaler("cuda", enabled=self.config.use_amp)

with torch.autocast(device_type="cuda", dtype=torch.float16, enabled=self.config.use_amp):
    loss = ...
    scaler.scale(loss).backward()
    scaler.step(self.optimizer)
    scaler.update()
```

```
# self_play.py / play_ai.py
self.model.eval()
with torch.inference_mode():
    root, mcts_info = MCTS(self.config).run(...)
```

另外，`checkpoint_interval=5` 需要立刻改。现在训练器每 5 step 就会把最新权重与 optimizer state 拷贝进 shared storage，且若 `save_model=True` 还会落盘。对任何非玩具实验，这都太频繁了。更合理的默认值应是按 **wall-clock 或 500~2000 step** 保存一次；评估最优 checkpoint 则单独保存。原配置与训练保存逻辑就在 `games/td_muzero.py` 和 `trainer.py`。¹ ³

同时，如果你暂时只做单机单卡研究，我甚至建议把“频繁 `ray.get(shared_storage.get_info.remote(...))` 的轮询设计”简化成更粗粒度的 actor 通信或者本地队列，因为 Ray 官方明确提示小任务会受调度开销影响，并把“在循环里频繁 `ray.get`”列为 anti-pattern。

³¹

算法升级路线

在工程基线修好之后，我建议你在算法上采取“两条路线并行，但优先一条落地”的策略。

务实路线：先做 **EfficientZero-lite + Gumbel + ReZero**。这样做的理由很简单。EfficientZero 已在 Atari 100k 上给出极强的样本效率，官方开源仓库也明确建议使用 torch AMP 提速；Gumbel MuZero 在低模拟数下更稳定、更强；ReZero 则直接瞄准了 reanalyse 的墙钟开销。把这三者的思想合在塔防上，通常比“直接把 MuZero 换成大 Transformer”更稳、更快见效。³²

冲高路线：在上面这条路线稳定后，再上 **UniZero / GTrXL**。UniZero 的核心价值不只是“换成 transformer”，而是把 latent state 与 latent history 解耦，用 attention 处理长时依赖，并在单帧输入下也能达到或超过传统多帧 MuZero-style baseline；GTrXL 则提供了 RL 里更稳定的 transformer 配方。对于塔防这种“当前收益取决于很久以前建了什么塔”的任务，这类模型有理论和经验上的吸引力。³³

如果你想少走弯路，我会给一个比较明确的结论：**要想尽快接近开源 MuZero 系列 SOTA，最值得认真考虑的是把环境迁到 LightZero 基座**。LightZero 已经统一集成了 MuZero、Sampled、Stochastic、EfficientZero、Gumbel、UniZero、ReZero 等路线，且 README 明确说明其 MCTS 有 Python/C++ 两套实现、并用混合异构编程加速最耗时的部分；它甚至有完整中文 README。对你这个项目来说，迁移成本虽然不低，但它会一次性解决 benchmark、MCTS 实现、训练框架、日志和后续对比实验的大量重复劳动。³⁴

实验设计、时间线与复现方案

要把“改造后接近 SOTA”说服人，实验设计必须同时覆盖**标准 benchmark**与**自定义任务**。因为这个塔防环境没有公开 leaderboard，最合理的实验组合是“双轨制”。

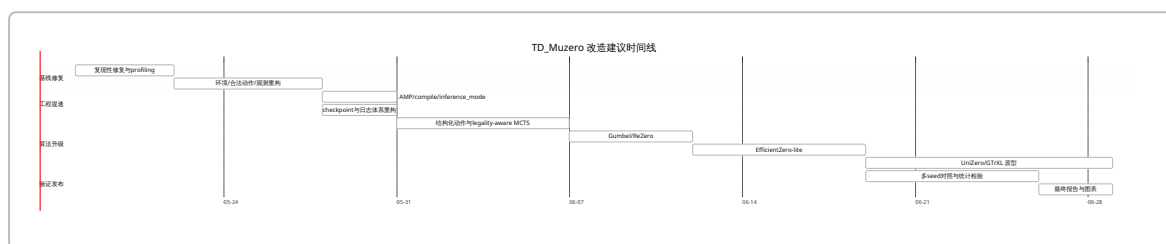
一条是**标准化验证轨**：选择 3 到 5 个公开、动作结构接近或 planning 价值明确的任务，优先推荐 `Pong / Breakout / MsPacman / LunarLander / MiniGrid-Memory` 这类有现成 MuZero/EfficientZero/UniZero 对照的环境；若预算足够，再做 Atari 100k 子集甚至 26 游戏套件。另一条是**塔防主任务轨**：以当前 `TD_Muzero` 环境为主，和“修复后旧 MuZero 基线”“PPO”“DreamerV3 或轻量世界模型基线”“LightZero-MuZero / EfficientZero”对比。EfficientZero 与 EfficientZero V2 分别代表了 Atari 100k 与通用 limited-data 设置下的强 baseline；UniZero 则代表长时依赖方向的上限路线。 ³⁵

建议实验矩阵

组别	改动组合	目的	主要指标	建议 seed 数
Baseline	原仓库，仅修复 reproducibility bug	形成可信基线	return、wave、base HP、invalid action、SPS	5
Speed	Baseline + 观测/合法动作重构 + AMP + compile + checkpoint 重构	看纯工程加速收益	SPS、ms/move、GPU util、same-return wall-clock	5
Action	Speed + 结构化动作头 + legality-aware latent MCTS	看策略与搜索质量收益	return、invalid action、policy entropy、sample efficiency	5
Search	Action + Gumbel / progressive simulation	看低模拟数下质量-速度折中	return @ 8/16/32 sims、wall-clock	5
SampleEff	Search + EfficientZero-lite / consistency loss	看样本效率提升	AUC(return vs env steps)、100k/500k step performance	5
LongHorizon	Search + UniZero/GTrXL backbone	看长时依赖收益	late-wave stability、memory proxy tasks、AUC	5
Final	最优组合，与 PPO / LightZero-MuZero / EfficientZero 对照	做最终结论	全指标 + 95% CI	10

显著性检验建议至少做两层：一层对每个 seed 的最终分数做 **Welch t-test**；另一层做 **bootstrap 95% CI**，尤其适合 episode return 这种重尾分布。对 wall-clock 和 SPS，也应把每个 seed 当独立样本报告均值与置信区间。因为当前仓库没有系统的统计检验逻辑，这一套最好独立加到 `scripts/aggregate_results.py` 中，而不是仅依赖 TensorBoard 曲线。 ⁸

建议时间线与资源



资源上，最低可行配置是 1 张 24GB 级 GPU + 16 核 CPU + 64GB RAM；更舒服的是 1×A100/H100 或 2×4090 + 32 核 CPU + 128GB RAM。原因很直接：这个仓库当前最重的不是单纯 GPU 算力，而是 CPU 环境步进 + MCTS Python 循环 + Ray actor 协调。所以 CPU 核数、单核性能与内存一样重要。若迁移到 LightZero 并启用 C++ MCTS，这个权重会更偏向 GPU。 36

可复现命令与日志策略

当前仓库已经提供了 `play_ai.py`，其说明中给出了推理用法：`python play_ai.py <checkpoint_dir>`。这可以保留，但训练与评估最好新增显式脚本，而不是沿用交互式入口。 11 37

建议新增下面三类命令：

```
# 训练
python -m scripts.train_td \
  --config configs/td_muzero_baseline.yaml \
  --seed 0 \
  --results_dir results/td_baseline_s0

# 批量评估
python -m scripts.eval_td \
  --checkpoint results/td_baseline_s0/best.pt \
  --episodes 200 \
  --mcts_simulations 64 \
  --save_json results/td_baseline_s0/eval_64sims.json

# 汇总多seed结果
python -m scripts.aggregate_results \
  --glob "results/td_*_eval_64sims.json" \
  --out_csv results/summary.csv \
  --out_fig_dir results/figures
```

日志/检查点策略建议改成下面的原则：每次运行保存 `config.yaml`、git SHA、依赖版本、环境 seed list、训练日志、profiling trace、`latest.pt`、`best_reward.pt`、`best_wallclock.pt`；Replay Buffer 单独存储，不要每次 checkpoint 都强耦合保存。这样既能避免 `checkpoint_interval=5` 带来的 IO 问题，也方便后期复现实验。原仓库已经有 `SharedStorage.save_checkpoint` 与 replay buffer 落盘逻辑，但频率与组织方式都需要重构。 10 8

建议最终至少输出以下图表：

图表类型	用途
return vs environment steps 折线图	样本效率主图
return vs wall-clock hours 折线图	工程优化是否真的提速
SPS / moves-per-second 柱状图	环境与搜索吞吐对比
invalid-action rate 折线图	legality-aware 改造收益
wave reached / final base HP 箱线图	塔防任务稳定性
不同 simulation 数下的 reward-速度 Pareto 图	Gumbel / progressive simulation 价值
profiler flame graph / timeline	CPU/GPU 热点定位

工程优化、风险与结论性建议

训练与推理速度优化建议

最确定的工程提速建议有四类。

第一类是 **PyTorch 原生加速**。AMP 文档明确指出，很多线性层和卷积在低精度下更快；`torch.autocast` + `GradScaler` 是官方混合精度训练路径；`inference_mode` 比 `no_grad` 更少额外开销；`torch.compile` 会对 compiled region 的 frame 进行缓存；activation checkpointing 用计算换显存；Profiler 可以导出 TensorBoard trace。对这个仓库，训练器与 self-play 推理器都适合吃这套增益。 38

第二类是 **MCTS 与环境 CPU 路径优化**。从工程优先级看，应该先做：合法动作缓存、观测向量化、减少 Python dict/list 构造、批量 recurrent inference、减少 `ray.get` 次数、放宽 checkpoint 间隔、把 `play_ai.py` 中重复的最小 MCTS 逻辑收敛回共享实现。若愿意接受更多重构，则直接迁到 LightZero 的 C++ `ctree` 路线，或者把当前 `Node/MCTS` 改写成 Cython/CPP。LightZero 官方 README 已明确把 Python/C++ 双实现和混合异构加速当成其核心卖点。 4 11 39

第三类是 **导出与量化**。如果最终要部署 `play_ai.py` 那种“看模型玩”的推理模式，ONNX Runtime 文档说明其支持 8-bit 量化，提供预处理、图优化、静态/动态量化与调试；并建议 transformer 模型优先动态量化、CNN 模型优先静态量化；CPU 上默认优先考虑 `S8S8 + QDQ`；对于 transformer，还有 QAttention 等专门优化。对于你这个项目，若最终仍是 CNN/轻量 conv backbone，可以优先走 **静态 INT8**；若转成 UniZero/Transformer，则优先尝试 **动态 INT8** 或混合精度 ONNX。 40

第四类是 **profiling 纪律**。Ray 文档建议用 Timeline、Dashboard flame graph、cProfile、PyTorch Profiler 和 Nsight；尤其强调小任务与频繁 `ray.get` 会破坏并行性。对这个仓库，我建议每次大改动都固定做三件事：导出一次 Ray timeline、抓一次 200 step 的 PyTorch trace、抓一次 env-only cProfile。否则你很可能“以为是模型慢”，实际上是 `legal_actions()`、checkpoint 或 Ray actor 轮询在拖后腿。 41

风险、替代方案与调试建议

最大的风险不是“改不出更高分”，而是**同时改太多导致你根本不知道哪一项有效**。所以所有改造都应遵循“先 correctness，再 interface，再 systems，再 algorithm”的顺序。尤其不要在现有环境表示、合法动作逻辑和 hook bug 都没清干净之前，就开始比较 transformer 与 conv 的优劣。那样会浪费很多 GPU 时数。

第二个风险是 **把自定义塔防任务误当成标准 benchmark**。这个环境目前没有公开 leaderboard，所以你最终最强的论文式结论应当写成：“在标准 benchmark 上复现接近 SOTA 的 MuZero-family 表现；在自定义 TD 环境中显著超过原始仓库与强基线。”这样最严谨，也最容易说服他人。

第三个风险是 **走过于激进的 Transformer/MoE 路线**。UniZero、ScaleZero、TransZero 都很有吸引力，但它们更适合作为第二阶段甚至第三阶段研究题。对当前仓库来说，真正高回报、低风险的前几步仍然是：环境改造、结构化动作、legality-aware search、Gumbel/ReZero、AMP/compile/C++ MCTS。 42

调试上，我会特别建议你加四个自检断言：

- **合法动作一致性检查**：在真实环境根节点与 latent node 上，统计 top-k 动作里非法动作比例；
- **观测完备性检查**：随机抽 100 个状态，确认 tower type / level / cooldown / wave 是否真的进入模型输入；
- **搜索质量检查**：记录不同 simulation 数下 root policy 的 KL 和最终 reward；
- **训练缩放检查**：在修复 hook 前后，对同一 batch 的梯度范数分布做对比。

结论性建议

如果目标是“把这个仓库改造成 SOTA，并尽可能提升训练与推理速度”，我的最终建议是：

首先，把现有仓库修成一个**正确、可复现、可测速**的版本：修随机种子、删重复初始化、修 hook bug、把 checkpoint 周期从 5 改成 500~2000、引入 AMP/ `compile` / `inference_mode` / Profiler；与此同时，立即重做 observation 与合法动作生成，并把 latent MCTS 从“全动作展开”改成“带 legality mask 的搜索”。这些改造不华丽，但会决定你的上限。 2 3 1 43

然后，在算法升级上采用“**EfficientZero-lite + Gumbel + ReZero**”优先，**UniZero 其次**的路线。前者更符合当前仓库的算力-收益比，也更容易在短期内把自定义 TD 任务做强；后者则适合作为追求更高上限和更长时依赖建模能力的第二阶段计划。 44

最后，从工程 ROI 看，最值得认真比较的不是“继续在本仓库 patch 到底”，而是“**是否尽快把环境移植到 LightZero**”。如果你希望尽快拿到强基线、系统 benchmark、C++ MCTS 和完整中文文档支持，迁移到 LightZero 往往会更快达到“接近 SOTA”的研究状态；如果你希望完全掌控每一行实现并做任务定制化创新，那就保留本仓库，但务必按本报告给出的顺序重构。 39

开放问题与局限

本报告对性能瓶颈的判断主要来自**代码静态审查**，而不是你当前机器上的真实 profiler 日志；因此，诸如“CPU 百分之多少耗在 `legal_actions()`、Ray、checkpoint、还是模型前向”这类问题，仍需要你按建议补一轮 PyTorch/Ray profiling 后再精细化。 3 4 45

此外，本次检索没有发现该自定义 Tower Defense 环境的公开 leaderboard 或标准对照结果，因此“达到 SOTA”必须通过“标准 benchmark 近 SOTA + 自定义 TD 显著超越强基线”的组合方式来定义，而不能只在仓库内部单独比较。

① td_muzero.py

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/games/td_muzero.py

2 **engine.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/games/engine.py

3 **trainer.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/trainer.py

4 **self_play.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/self_play.py

5 **models.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/models.py

6 7 20 32 35 44 **<https://arxiv.org/abs/2111.00210>**

<https://arxiv.org/abs/2111.00210>

8 **muzero.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/muzero.py

9 **replay_buffer.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/replay_buffer.py

10 **shared_storage.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/shared_storage.py

37 **play_ai.py**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/play_ai.py

12 **README.md**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/README.md

13 **requirements.lock**

https://github.com/SSHion0721/TD_Muzero/blob/8f9fe0c4ac01472de4f3d0c42b832b0ae8790e73/muzero-general/requirements.lock

14 31 41 45 **<https://docs.ray.io/en/latest/ray-observability/user-guides/debug-apps/optimize-performance.html>**

<https://docs.ray.io/en/latest/ray-observability/user-guides/debug-apps/optimize-performance.html>

15 16 **<https://arxiv.org/abs/1911.08265>**

<https://arxiv.org/abs/1911.08265>

17 29 **<https://arxiv.org/abs/2104.06303>**

<https://arxiv.org/abs/2104.06303>

18 **<https://openreview.net/pdf?id=bERaNdognO>**

<https://openreview.net/pdf?id=bERaNdognO>

19 **<https://openreview.net/pdf?id=X6D9bAHhBQ1>**

<https://openreview.net/pdf?id=X6D9bAHhBQ1>

21 **<https://arxiv.org/abs/2403.00564>**

<https://arxiv.org/abs/2403.00564>

- 22 <https://arxiv.org/abs/2310.08348>
<https://arxiv.org/abs/2310.08348>
- 23 33 42 <https://ar5iv.org/abs/2406.10667>
<https://ar5iv.org/abs/2406.10667>
- 24 <https://ar5iv.org/abs/2404.16364>
<https://ar5iv.org/abs/2404.16364>
- 25 <https://arxiv.org/abs/2310.11305>
<https://arxiv.org/abs/2310.11305>
- 26 <https://arxiv.org/abs/1910.06764>
<https://arxiv.org/abs/1910.06764>
- 27 <https://arxiv.org/abs/2302.04798>
<https://arxiv.org/abs/2302.04798>
- 28 <https://arxiv.org/abs/2509.11233>
<https://arxiv.org/abs/2509.11233>
- 30 38 43 <https://docs.pytorch.org/docs/2.12/amp.html>
<https://docs.pytorch.org/docs/2.12/amp.html>
- 34 36 39 <https://github.com/opencv/opencv4>
<https://github.com/opencv/opencv4>
- 40 <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>